

제품 요구사항 문서 PRD

Qeploy Product Requirements Document

0. 문서 범위

이 문서는 Qeploy의 축소된 MVP가 아니라, 사용자가 구상한 **전체 제품 범위**를 기준으로 작성한다. Qeploy는 AI Agent 기반 개발, Docker 기반 미리보기, GitHub 기반 버전관리, GitHub Pages 및 개인 AWS/GCP 배포, 도메인 연결, 클라우드 인프라 설정 및 운영 관리까지 포함하는 웹서비스 운영 자동화 플랫폼이다.

1. 제품 개요

1.1 제품명

Qeploy

1.2 한 문장 정의

Qeploy는 비개발자와 입문 개발자가 AI Agent와 자연어로 웹 프로젝트를 생성·수정하고, Docker 기반 미리보기에서 실제 동작을 검증한 뒤, GitHub 기반 이력/버전 관리를 거쳐 GitHub Pages 또는 개인 AWS/GCP 같은 배포 환경에 배포하고, qeploy.com 서브도메인 또는 사용자 지정 도메인까지 연결할 수 있게 하는 AI Agent 기반 웹서비스 운영 자동화 플랫폼이다. AWS/GCP 연결 시에는 한 페이지 인프라 설정, 예상 요금 확인, 배포 이후 자연어 기반 인프라/서버 운영 관리까지 지원한다.

1.3 제품의 본질

Qeploy는 단순 AI 코드 생성 도구가 아니다. 사용자가 만든 결과물이 실제 서비스가 되기 위해 필요한 생성, 수정, 빌드, 미리보기, 버전관리, 배포, 도메인, 인프라 운영까지의 흐름을 자연어와 Agent 구조로 연결하는 제품이다.

2. 배경 및 문제 정의

2.1 배경

AI의 확산으로 비개발자와 입문 개발자도 웹서비스를 빠르게 만들 수 있게 되었다. 그러나 실제 서비스화 과정에는 여전히 많은 장벽이 남아 있다.

사용자는 웹사이트나 웹앱을 만들 수는 있지만, 이를 실제로 배포하고 운영하는 과정에서 다음 문제를 겪는다.

- 배포 방식에 대한 지식 부족
- GitHub, 브랜치, PR, merge, 버전관리 개념의 어려움
- Docker, 빌드, 런타임 환경에 대한 이해 부족
- 도메인 연결, DNS, HTTPS 설정의 어려움
- AWS/GCP 같은 클라우드 인프라 설정의 복잡성
- 예상 비용을 알기 어려운 문제
- 외주로 만든 서비스의 수정/운영을 다시 외주에 맡겨야 하는 이중 비용 문제

2.2 핵심 문제

비개발자와 입문 개발자는 AI를 통해 웹 결과물을 만들 수는 있지만, 이를 실제로 배포하고 운영 가능한 서비스로 만드는 과정에서 막힌다.

2.3 해결 방향

Qeploy는 사용자가 GitHub, Docker, 배포, DNS, AWS/GCP 같은 복잡한 기술 개념을 직접 이해하지 않아도 되도록 한다. 사용자는 자연어로 원하는 결과를 말하고, 시스템은 Agent 구조와 자동화 workflow를 통해 작업을 처리한다.

사용자가 반드시 알아야 하는 정보는 다음으로 제한한다.

- 미리보기 결과
- 승인창의 작업 요약
- 배포 URL
- 현재 버전
- 예상 비용

3. 대상 사용자

3.1 주 사용자

- 비개발자
- 입문 개발자

3.2 주요 사용 사례

- 랜딩페이지 제작
- 기업 홈페이지 제작
- 포트폴리오 사이트 제작
- 기존 ZIP 결과물 수정
- 기존 GitHub 레포 기반 수정/배포
- AI로 만든 웹 결과물 실제 서비스화
- 배포 후 도메인 연결
- AWS/GCP 인프라 설정 및 운영 관리

3.3 사용자의 기대

사용자는 다음을 기대한다.

- 코드나 인프라를 몰라도 자연어로 요청 가능
 - 실제 동작하는 미리보기 확인 가능
 - 승인 후 배포 URL 확보 가능
 - 도메인 연결 가능
 - AWS/GCP 설정 시 예상 요금 확인 가능
 - 배포 이후에도 자연어로 서버/인프라 운영 가능
-

4. 제품 목표

4.1 핵심 목표

Qeploy는 웹서비스 제작 이후 실제 서비스화와 운영까지의 과정을 단순화한다.

4.2 주요 목표

1. 비개발자도 배포 전문지식 없이 웹서비스를 배포할 수 있게 한다.
2. 입문 개발자가 GitHub, Docker, 배포, 도메인, 클라우드 운영을 자연어로 다룰 수 있게 한다.
3. AI가 생성/수정한 결과를 Docker 기반 미리보기에서 실제로 테스트할 수 있게 한다.
4. GitHub 기반 버전관리와 승인 기반 배포 흐름을 제공한다.
5. GitHub Pages, 개인 AWS, GCP 등 다양한 배포 환경을 지원한다.

6. qeploy.com 무료 서브도메인과 사용자 지정 도메인을 지원한다.
 7. AWS/GCP 연결 시 인프라 설정, 예상 요금, 운영 관리를 지원한다.
-

5. 핵심 차별점

Qeploy의 차별점은 단순 AI 코드 생성이 아니라, 생성·수정 이후 Docker 미리보기, GitHub 버전관리, 승인 기반 배포, 도메인 연결, 클라우드 인프라 설정 및 운영까지 이어지는 전체 웹서비스 운영 흐름을 자연어와 Agent 구조로 연결한다는 점이다.

5.1 기존 AI 코드 생성 도구와의 차이

일반 AI 코드 생성 도구는 코드 생성 또는 수정에 집중한다. Qeploy는 코드가 실제 서비스로 운영되기까지 필요한 흐름을 함께 다룬다.

5.2 기존 배포 플랫폼과의 차이

일반 배포 플랫폼은 사용자가 배포 설정을 직접 이해하고 구성해야 한다. Qeploy는 자연어 요청과 Agent 구조를 통해 배포 및 운영 흐름을 자동화한다.

6. 포함 범위 및 제외 범위

6.1 포함 범위

현재 PRD의 핵심 범위는 다음과 같다.

- GitHub 로그인 및 GitHub 레포 연결
- 프로젝트 생성
- ZIP 업로드
- 기존 GitHub 레포 불러오기
- AI Agent 기반 자연어 생성/수정
- Docker 컨테이너 기반 미리보기
- preview 브랜치 기반 작업 커밋
- PR 생성 및 main/production merge
- merge 기준 버전 관리
- GitHub Pages 배포
- 개인 AWS/GCP 배포

- 한 페이지 인프라 설정
- 예상 요금 확인
- geploy.com 무료 서브도메인 연결
- 사용자 지정 커스텀 도메인 연결
- 도메인 변경, DNS 확인, HTTPS 처리
- 배포 이후 자연어 기반 인프라/서버 운영 관리
- 승인 기반 작업 흐름
- Project Settings
- Overview 기반 운영 대시보드

6.2 제외 범위

다음 항목은 현재 핵심 PRD 범위에서 제외한다.

1. 트래픽 분석, GA4 연동, 방문자 수, 페이지뷰, 트래픽 추이 그래프
2. 팀/조직 권한, 대규모 협업, 엔터프라이즈 권한 체계
3. [Polar.sh](https://polar.sh) 등 결제/과금 도구의 구체적 연동 방식
4. LLM 모델 선택, 사용자 개인 LLM API Key 연결, 모델 라우팅 설정
5. 자체 클라우드 호스팅 플랫폼 사업 모델 구축
6. 외부 분석 도구 기반 운영 대시보드

위 항목들은 제품 핵심 흐름이 아니라 추후 확장 기능으로 분리한다.

7. 핵심 사용자 여정

7.1 전체 사용자 여정

1. 사용자는 GitHub로 로그인한다.
2. Projects 화면에서 새 프로젝트를 만들거나 ZIP을 업로드하거나 기존 GitHub 레포를 불러온다.
3. 프로젝트에 들어가면 Overview에서 현재 상태를 확인한다.
4. AI Workspace에서 자연어로 생성, 수정, 배포, 도메인, 인프라 운영 요청을 한다.
5. 의사결정 Agent가 요청을 해석하고 필요한 전문 Agent로 분기한다.

6. 코드 변경이 필요한 경우 preview 브랜치에 커밋한다.
 7. Docker 기반 미리보기에서 실제 설치·빌드·실행 결과를 확인한다.
 8. 사용자는 승인창에서 작업 요약을 확인하고 승인한다.
 9. 배포 요청 시 PR 생성, main/production merge, version 증가, 배포 workflow 실행이 진행된다.
 10. 배포 후 기본 URL, qeploy.com 서브도메인 또는 커스텀 도메인으로 접근한다.
 11. AWS/GCP 배포 프로젝트는 Overview와 AI Workspace를 통해 서버/인프라 운영 관리까지 수행한다.
-

8. 핵심 화면 구조

Qeploy의 핵심 화면 측은 다음 4개다.

1. Projects
 2. Overview
 3. AI Workspace
 4. Project Settings
-

9. Projects 화면

9.1 역할

Projects 화면은 서비스의 시작점이며, 사용자가 자신의 프로젝트 목록을 확인하고 새 프로젝트를 생성하거나 기존 결과물을 가져오는 화면이다.

9.2 프로젝트 카드 UI

프로젝트 목록은 카드형 UI로 제공한다.

각 카드에는 다음 정보를 표시한다.

- 프로젝트명
- 상태 배지
- 최근 수정일
- Live 상태일 때만 현재 URL

9.3 카드 동작

- URL 클릭 시 새 탭으로 실제 서비스가 열린다.
- URL을 제외한 카드 영역 클릭 시 Overview로 이동한다.
- 프로젝트명 클릭도 카드 전체 클릭과 동일하게 Overview로 이동한다.

9.4 정렬

프로젝트 목록은 최근 수정순으로 정렬한다.

최근 수정일은 GitHub와 유사한 상대시간 표기를 사용한다.

예:

- Updated yesterday
- Updated last week
- Updated 2 weeks ago
- 한 달 이상 지난 경우: Updated on Feb 17

9.5 프로젝트 생성/가져오기 진입점

Projects 화면에서는 다음 작업을 시작할 수 있어야 한다.

- 새 프로젝트 생성
- ZIP 업로드
- GitHub 레포 불러오기

10. 프로젝트 생성 및 가져오기

10.1 새 프로젝트 생성

사용자는 빈 프로젝트로 시작할 수 있다.

새 프로젝트는 생성 직후 반드시 GitHub 레포와 동일한 객체일 필요는 없다. 프로젝트와 GitHub 레포는 병렬 관계로 관리된다.

사용자는 이후 다음 중 하나를 선택할 수 있다.

- 기존 GitHub 레포 연결
- 새 GitHub 레포 생성 후 연결

10.2 ZIP 업로드

사용자는 기존 결과물을 ZIP 파일로 업로드해 프로젝트로 가져올 수 있다.

ZIP 업로드 후에도 GitHub 레포 연결이 필요하다.

10.3 GitHub 레포 불러오기

사용자는 접근 가능한 GitHub 레포 목록을 보고 하나를 선택할 수 있다.

흐름:

1. GitHub 레포 목록 표시
2. 사용자가 레포 선택
3. 가져오기 확인 단계 표시
4. 확인 후 프로젝트 목록에 추가

레포 목록에는 기본적으로 레포 이름만 표시한다.

11. GitHub 역할 및 권한 정책

11.1 GitHub의 역할

GitHub는 다음의 중심 역할을 한다.

- 로그인
- 소스 저장소
- 작업 이력
- 브랜치
- 커밋
- PR
- merge
- 버전 관리
- GitHub Pages 배포 또는 workflow 실행

11.2 프로젝트와 레포 관계

서비스 내부 프로젝트와 GitHub 레포는 동일 객체가 아니라 연결 가능한 병렬 객체다.

가능한 흐름:

- 빈 프로젝트 생성 후 기존 레포 연결
- 빈 프로젝트 생성 후 새 레포 생성 및 연결

- GitHub 레포 불러오기 후 프로젝트로 추가
- ZIP 업로드 후 새 레포 생성 또는 기존 레포 연결

11.3 GitHub 로그인

GitHub 로그인 필수다.

AWS/GCP 연결은 필수가 아니며, 사용자가 개인 클라우드 배포를 선택할 때 추가로 연결한다.

11.4 권한 정책

Qeploy는 다음 작업을 위해 필요한 GitHub 권한을 요청한다.

- 레포 생성
- 기존 레포 연결
- 브랜치 생성/관리
- 커밋
- PR 생성
- merge
- GitHub Pages 배포
- workflow 실행

단, Qeploy는 사용자가 서비스에서 생성한 레포 또는 명시적으로 연결한 레포에 대해서만 권한을 사용한다. 사용자가 선택하지 않은 다른 레포에는 접근하지 않는 것을 원칙으로 한다.

12. Agent 구조

12.1 개요

Qeploy는 단일 Agent가 모든 작업을 직접 수행하는 구조가 아니다. 사용자의 모든 자연어 요청은 먼저 의사결정 Agent를 통과하고, 이후 필요한 전문 Agent로 분기된다.

사용자는 내부 Agent 분기 과정을 보지 않는다. 사용자는 자연어 입력과 승인창만 사용한다.

12.2 Agent 역할

1. 의사결정 Agent

역할:

- 사용자의 자연어 요청 해석
- 대화 맥락 이해
- 프로젝트 상태 반영
- 작업 의도 분류
- 전문 Agent로 라우팅

분류 범주:

1. 코드 관련
2. 배포 관련
3. 도메인 관련
4. 인프라/운영 관련

2. 코드 관련 Agent

역할:

- 프로젝트 생성
- 코드 수정
- UI 변경
- 기능 추가
- 기능 삭제
- 오류 수정
- 빌드 실패 원인 분석 및 수정 제안

3. 배포 관련 Agent

역할:

- GitHub Pages 배포
- AWS/GCP 배포
- 고정 function 실행
- 기존 workflow 실행
- 신규 workflow 생성

- 배포 버전 선택
- 특정 버전 재배포

4. 도메인 관련 Agent

역할:

- qeploy.com 서브도메인 연결
- 사용자 지정 커스텀 도메인 연결
- 도메인 변경
- DNS 설정 확인
- HTTPS 처리

5. 인프라/운영 관리 Agent

역할:

- AWS/GCP 인프라 수정
- 서버 상태 확인
- 로그 확인
- 장애 원인 분석
- 서버 재시작
- 리소스 조정
- 비용 최적화 제안
- 불필요 리소스 정리

13. AI Workspace

13.1 역할

AI Workspace는 사용자가 자연어로 프로젝트 생성, 수정, 배포, 도메인 연결, 인프라 운영 요청을 하는 핵심 작업 화면이다.

13.2 화면 구성

AI Workspace는 다음을 포함한다.

- Docker 기반 미리보기 패널

- AI 대화창
- 자연어 입력창
- 승인 상태/반영 상태 표시
- 변경 요약
- 필요 시 diff 보기
- 오류 패널
- 물음표 도움말 아이콘
- 대화 세션 목록
- 새 대화 시작

13.3 입력 방식

별도 액션 칩은 제공하지 않는다.

사용자는 자연어만으로 여러 범주의 작업을 한 번에 요청할 수 있다.

예:

- 헤더 크기를 줄이고 GitHub Pages로 배포해줘.
- 카페 랜딩페이지를 만들고 cafe.qeploy.com으로 연결해줘.
- AWS로 배포하고 예상 비용이 3만원 이하가 되도록 설정해줘.
- 서버 상태 확인하고 문제가 있으면 원인을 알려줘.

13.4 Placeholder

입력창에는 다음과 같은 placeholder를 제공한다.

┃ 원하는 작업을 자연어로 입력하세요. 예: 카페 랜딩페이지를 만들고 배포해줘

13.5 대화 세션

AI Workspace의 대화는 세션 단위로 저장된다.

사용자는 다음을 할 수 있다.

- 기존 대화 이어가기
- 새 대화 시작
- 대화 세션 삭제

대화 목록에는 다음을 표시한다.

- 대화 제목
- 마지막 수정시간

정렬 기준은 최근 수정순이다.

새 대화 제목은 자동 생성된다.

기존 대화를 다시 열면 이전 대화 전체를 불러와 이어서 진행할 수 있어야 한다.

13.6 대화 세션 삭제 및 휴지통

대화 세션 삭제 시 별도 확인 모달 없이 휴지통으로 이동한다.

휴지통 정책:

- 계정 전체 공용 휴지통 사용
- 기본 대화 목록에서는 숨김
- 휴지통에서는 대화 제목, 프로젝트명, 남은 보관일 표시
- 복구 가능
- 즉시 삭제 가능
- 7일 후 자동 완전 삭제
- 연결 가능한 프로젝트가 없는 대화 세션은 복구 불가

13.7 오버레이 튜토리얼

새 대화 최초 진입 시 오버레이 튜토리얼을 1회 표시한다.

오버레이는 단계형 튜토리얼이 아니라, 화면의 핵심 요소를 화살표/하이라이트로 짚는 방식이다.

포함 내용:

- AI Agent 입력창 설명
- Docker 기반 미리보기 영역 설명
- 승인 흐름 설명
- 도움말 및 주요 버튼 설명
- 플메시지 예시 2~3개

닫기 버튼은 두지 않고, 화면 클릭 시 사라지도록 한다.

이후 물음표 아이콘을 클릭하면 동일한 오버레이 튜토리얼을 다시 볼 수 있다.

14. Docker 기반 미리보기

14.1 정의

미리보기는 Docker 컨테이너 기반 실행 환경으로 제공된다.

프로젝트는 컨테이너 안에서 실제 설치, 빌드, 실행되며, 사용자는 외부 URL이 아닌 서비스 내부 미리보기 패널에서 실제 동작을 테스트한다.

14.2 주요 원칙

- 단순 정적 파일 렌더링이 아니다.
- 실제 프로젝트 실행 환경을 컨테이너 안에 구성한다.
- 클릭, 입력, 화면 이동 등 실제 사용 테스트가 가능해야 한다.
- 외부 공개 preview URL은 기본적으로 발급하지 않는다.
- 서비스 내부 워크스페이스에서만 접근한다.
- 컨테이너는 프로젝트 또는 작업 세션 단위로 생성된다.
- 미사용 시 자동 종료 및 정리된다.
- 반복 작업 시 의존성 캐시, 빌드 캐시, 이미지 캐시를 활용한다.

14.3 지원 확장성

Docker 기반 미리보기 구조를 사용하기 때문에 최종적으로 다음 프로젝트 유형을 지원할 수 있어야 한다.

- 정적 HTML/CSS/JS
- React
- 프론트엔드 프로젝트
- 백엔드 프로젝트
- 풀스택 프로젝트
- API 서버
- 서버리스 함수
- 데이터베이스가 필요한 웹서비스

15. 작업/버전 관리 흐름

15.1 기본 흐름

사용자의 자연어 요청 이후 작업 흐름은 다음 순서를 따른다.

1. 자연어 요청
2. 의사결정 Agent 해석
3. 전문 Agent 처리
4. preview 브랜치에 요청 단위 커밋
5. Docker 기반 미리보기 생성
6. 사용자 승인
7. 배포 대기 상태
8. 배포 요청
9. PR 생성
10. main/production merge
11. 버전 증가
12. 배포 workflow 실행

15.2 preview 브랜치

코드 변경은 preview 브랜치에 사용자 요청 단위로 커밋된다.

변경 승인 후에도 즉시 main/production에 반영되지 않는다. 배포 요청 시점에 PR 생성 및 merge가 진행된다.

15.3 버전 번호 정책

merge가 발생할 때마다 버전 번호가 증가한다.

예:

- 첫 번째 merge: v1
- 두 번째 merge: v2
- 세 번째 merge: v3

15.4 특정 버전 배포 및 롤백

특정 버전 배포와 롤백은 merge 기준 버전 이력을 기반으로 처리한다.

사용자는 merge 이력에서 다음 정보를 확인할 수 있다.

- 버전 번호
 - 커밋 해시
 - 작업 요약
 - 날짜시간
-

16. 승인 구조

16.1 승인 대상

기본 승인 대상은 다음과 같다.

1. 개발/수정
2. 배포
3. 도메인 연결
4. 인프라 운영

16.2 개발/수정 Approval

대상:

- 코드 수정
- UI 변경
- 기능 추가
- 기능 삭제
- 실제 변경사항 반영

16.3 배포 Approval

대상:

- GitHub Pages 배포
- AWS/GCP 배포
- 특정 버전 재배포
- workflow 실행

16.4 도메인 연결 Approval

대상:

- geploy.com 서브도메인 연결
- 사용자 지정 도메인 연결
- 도메인 변경
- DNS/HTTPS 관련 실제 반영

16.5 인프라 운영 Approval

대상:

- 서버 재시작
- 리소스 스펙 변경
- 오토스케일링 변경
- 비용 증가 가능성이 있는 변경
- 서비스 중단/영향 가능성이 있는 작업

16.6 승인 설정 위치

승인 On/Off는 Project Settings의 Chat Settings에서 관리한다.

16.7 승인창 표시 원칙

의도 파악 결과 자체는 사용자에게 중간 노출하지 않는다.

사용자는 승인창에서 최종 정리된 작업 결과만 확인한다.

승인창에는 각 작업을 한 줄 요약 중심으로 보여준다.

17. 배포 구조

17.1 배포 대상

배포는 GitHub Pages에 한정되지 않는다.

지원 배포 환경:

- GitHub Pages
- 개인 AWS
- 개인 GCP

17.2 GitHub Pages 배포

GitHub Pages는 정적 프론트엔드 프로젝트를 빠르게 배포하는 기본 경로다.

17.3 개인 AWS/GCP 배포

사용자가 개인 AWS/GCP 계정을 연결하면, Qeploy는 서비스 내부의 한 페이지 인프라 설정 화면을 통해 배포 구성을 제공한다.

사용자는 다음을 설정할 수 있어야 한다.

- 클라우드 제공자
- 연결 계정
- 리전
- 배포 아키텍처
- 컴퓨팅 리소스
- 스토리지
- 네트워크 설정
- 오토스케일링
- 로드밸런서
- 데이터베이스 연결

선택값에 따라 예상 요금이 표시된다.

사용자는 예산에 맞게 인프라 설정을 조정한 뒤 배포를 승인한다.

17.4 배포 Agent 역할

배포 관련 Agent는 선택한 배포 환경에 따라 다음을 수행한다.

- 고정 function 실행
- 기존 workflow 실행
- 신규 workflow 생성
- IaC 또는 자동화 구성 실행
- 배포 결과 확인
- 배포 실패 시 원인 분석

18. 배포 승인창

18.1 기본 문구

배포 요청 시 기본 문구는 다음과 같다.

| 최신 승인 버전으로 배포할까요?

18.2 표시 정보

배포 승인창에는 다음을 표시한다.

- 배포할 버전
- 배포에 사용할 환경 또는 대상
- 배포 후 예상 URL
- 변경 요약

각 항목은 한 줄 요약 중심으로 표시한다.

18.3 버전 선택

사용자가 원하면 merge 이력 기준으로 이전 버전 목록을 확인할 수 있다.

사용자는 특정 버전을 선택해 배포하거나 롤백할 수 있다.

19. 도메인 구조

19.1 접근 URL 유형

Qeploy의 도메인/URL 구조는 3가지로 구성된다.

1. 배포 환경 기본 URL
2. geploy.com 무료 서브도메인
3. 사용자 지정 커스텀 도메인

19.2 배포 환경 기본 URL

커스텀 도메인을 연결하지 않아도 배포 환경이 기본 제공하는 URL로 접근할 수 있다.

예:

- GitHub Pages 기본 URL
- AWS 기본 엔드포인트 또는 로드밸런서 URL
- GCP 기본 서비스 URL

19.3 geploy.com 무료 서브도메인

Qeploy는 무료 관리형 서브도메인을 제공한다.

예:

- cafe.qeploy.com
- portfolio.qeploy.com

서브도메인은 랜덤 생성하지 않는다.

사용자가 직접 원하는 이름을 입력한다.

시스템은 중복 여부를 확인하고, 사용 가능하면 연결 승인창을 보여준다.

19.4 사용자 지정 커스텀 도메인

사용자는 직접 보유한 도메인을 연결할 수 있다.

예:

- cafe.kr
- brand.com

도메인 관련 Agent는 DNS 설정 확인, HTTPS 처리, 연결 검증을 지원한다.

20. 도메인 승인창

도메인 연결 요청 시 시스템은 사용자가 지정한 도메인 또는 qeploy.com 서브도메인을 기준으로 중복/사용 가능 여부와 연결 가능 상태를 확인한다.

사용 가능하면 승인창에 다음을 표시한다.

- 연결할 도메인
- 연결 대상 배포 환경
- HTTPS 처리 여부
- 예상 적용 결과

승인 후 도메인 관련 Agent가 DNS/HTTPS/연결 작업을 수행한다.

21. 클라우드 인프라 운영

21.1 정의

AWS/GCP 배포는 배포에서 끝나지 않는다. Qeploy는 배포 이후 클라우드 인프라와 서버 운영까지 자연어로 관리할 수 있어야 한다.

21.2 지원 작업

사용자는 자연어로 다음 작업을 요청할 수 있다.

- 서버 상태 확인
- 인프라 수정
- 리소스 조정
- 로그 확인
- 장애 원인 분석
- 재시작
- 재배포
- 비용 최적화
- 불필요 리소스 정리

21.3 승인 원칙

비용 증가 또는 서비스 영향이 있는 작업은 승인 대상이다.

22. Overview

22.1 역할

Overview는 프로젝트 진입 시 첫 화면이다.

Overview는 단순 요약 화면이 아니라, Operations Dashboard 역할을 겸한다.

22.2 표시 정보

Overview에는 다음을 표시한다.

- 현재 URL
- URL 복사 버튼
- 배포 상태 배지
- 현재 배포 버전
- 최근 반영 이력 3개
- 가장 최근 커밋 이력
- 운영 조치 진입점
- Open AI Agent 버튼
- Project Settings 버튼

- 프로젝트 제거 진입점

트래픽/분석 기능은 현재 범위에서 제외한다.

22.3 현재 URL

현재 URL은 가장 우선 노출한다.

클릭 시 새 탭으로 열린다.

URL 옆에는 복사 버튼을 제공한다.

22.4 배포 상태 배지

배포 상태는 다음 배지 체계를 사용한다.

- Live
- Pending
- Failed
- Draft

22.5 현재 배포 버전

현재 배포 버전은 배포 상태와 같은 줄에 라벨 형태로 표시한다.

예:

| Current Version: v12

22.6 최근 반영 이력

최근 반영 이력은 3개까지 표시한다.

각 항목에는 다음을 표시한다.

- 작업 요약
- 날짜시간
- 배포 여부

22.7 가장 최근 커밋 이력

가장 최근 커밋 이력은 다음 형식으로 표시한다.

- 커밋 메시지
- 해시

- 상대시간
-

23. Project Settings

23.1 정의

Project Settings는 프로젝트의 기본값과 정책을 관리하는 곳이다.

작업 실행 화면이 아니라, 프로젝트가 어떤 이름, 연결, 버전 정책, 배포 기본값, 도메인 정책, 클라우드 설정, 비용 기준, 환경값, Chat 승인 정책, 위험 작업 기준으로 동작할지 정하는 설정 허브다.

23.2 최종 섹션

Project Settings는 다음 10개 섹션으로 구성된다.

1. General
2. Repository
3. Version & Release Policy
4. Deployment Defaults
5. Domain & HTTPS
6. Cloud Infrastructure
7. Cost & Budget
8. Environment & Secrets
9. Chat Settings
10. Danger Zone

23.3 General

- 프로젝트명: 서비스 안에서 보여지는 프로젝트 이름
- 프로젝트 설명: 이 프로젝트가 무엇을 위한 것인지 적는 간단한 설명
- 생성일: 프로젝트가 처음 만들어진 날짜

23.4 Repository

- 연결된 GitHub 레포: 현재 프로젝트와 연결된 GitHub 저장소
- 기존 레포 연결: 비어 있는 프로젝트에 기존 GitHub 레포를 연결

- 새 레포 생성 후 연결: 새 GitHub 레포를 만들고 프로젝트에 연결
- 레포 연결 해제: 프로젝트와 GitHub 레포의 연결만 끊음

23.5 Version & Release Policy

- PR 생성 정책: 배포 시 PR을 자동으로 만들지에 대한 규칙
- Merge 정책: 어떤 시점에 main/production으로 병합할지에 대한 규칙
- 버전 번호 정책: merge 기준으로 v1, v2처럼 버전을 증가시키는 규칙
- 롤백 기준: 어떤 이력을 기준으로 이전 버전으로 되돌릴지 정하는 값

23.6 Deployment Defaults

- 기본 배포 방식: 기본으로 사용할 배포 환경
- 기본 배포 버전: 기본적으로 최신 승인 버전을 배포할지 정하는 값
- 자동 배포 여부: 특정 조건에서 자동 배포를 허용할지 여부
- 수동 배포 여부: 사용자가 명시적으로 요청해야 배포되는 방식인지 여부
- 배포 후 URL 표시 방식: 배포 완료 후 URL을 어떤 방식으로 보여줄지
- 배포 실패 처리: 실패 시 원인 분석, 수정 제안, 재시도 흐름을 어떻게 둘지

23.7 Domain & HTTPS

- 기본 배포 URL: 도메인을 연결하지 않아도 접근 가능한 기본 URL
- 커스텀 도메인: 사용자가 직접 연결한 도메인
- 도메인 연결 상태: 도메인이 정상 연결됐는지 확인하는 상태값
- 도메인 변경: 기존 도메인을 다른 도메인으로 바꾸는 기능
- HTTPS 적용 상태: HTTPS가 적용되어 있는지 보여주는 값
- HTTPS 강제 적용: HTTP 접속을 HTTPS로 자동 전환할지 여부
- DNS 설정 상태: 도메인의 DNS 설정이 올바른지 확인하는 값
- www 리디렉션: www.example.com과 example.com을 어떻게 처리할지 정하는 옵션

23.8 Cloud Infrastructure

- 클라우드 제공자: AWS 또는 GCP 중 어떤 클라우드를 사용할지
- 연결된 클라우드 계정: 배포와 인프라 관리를 수행할 사용자 클라우드 계정

- 리전: 서버나 리소스를 어느 지역에 둘지 선택
- 배포 아키텍처: 서버, 컨테이너, 서버리스 등 운영 방식
- 컴퓨팅 리소스: 서버 스펙이나 실행 리소스 크기
- 스토리지: 파일, 이미지, 데이터 저장에 사용할 리소스
- 네트워크 설정: 방화벽, 접근 설정 등 네트워크 관련 값
- 오토스케일링: 트래픽에 따라 리소스를 자동으로 늘릴지 여부
- 로드밸런서: 여러 서버로 트래픽을 분산할지 여부
- 데이터베이스 연결: DB가 필요한 경우 어떤 DB와 연결할지
- 현재 인프라 상태: 실행 중, 중지됨, 오류 등 현재 리소스 상태

GitHub Pages 프로젝트에서도 Cloud Infrastructure 섹션은 숨기지 않는다. 단, 비활성화 색상과 클릭 불가 상태로 표시한다.

23.9 Cost & Budget

- 예상 월 비용: 현재 인프라 설정 기준으로 예상되는 월 비용
- 리소스별 예상 비용: 서버, 스토리지, 네트워크 등 항목별 예상 비용
- 예산 설정: 사용자가 감당 가능한 월 예산
- 예산 초과 경고: 예상 비용이 예산을 넘을 때 보여주는 경고
- 트래픽 기준 비용 변화: 사용량이 늘어날 때 비용이 어떻게 변할지 보여주는 값
- 비용 최적화 제안: 더 저렴하게 운영할 수 있는 설정 추천
- 불필요 리소스 탐지: 사용하지 않는 클라우드 리소스가 있는지 확인

GitHub Pages 프로젝트에서도 Cost & Budget 섹션은 숨기지 않는다. 단, 클라우드 인프라가 연결되지 않은 경우 비활성화 상태로 표시한다.

23.10 Environment & Secrets

- 환경변수: 코드 밖에서 관리하는 설정값
- API URL: 프론트엔드가 데이터를 요청할 서버 주소
- Secret Key: 외부에 노출되면 안 되는 민감한 키
- Preview 환경값: 미리보기에서만 사용하는 환경 설정
- Production 환경값: 실제 배포 환경에서 사용하는 환경 설정

- 값 숨김 처리: 민감한 값을 화면에 그대로 노출하지 않는 기능
- 변경 이력: 환경값이나 Secret이 언제 바뀌었는지 기록

23.11 Chat Settings

Chat Settings는 AI Workspace에서 사용자의 자연어 요청을 처리할 때 적용되는 프로젝트별 대화/승인 정책을 관리한다.

포함 항목:

- 개발/수정 Approval: 코드 수정, 기능 추가, UI 변경을 승인 후 반영할지 여부
- 배포 Approval: 배포 실행 전에 승인창을 띄울지 여부
- 도메인 연결 Approval: 도메인 연결이나 변경 전에 승인창을 띄울지 여부
- 인프라 운영 Approval: 서버 재시작, 리소스 변경 등 클라우드 운영 작업에 승인창을 띄울지 여부
- 프로젝트별 Chat 지침: 이 프로젝트에서 AI가 참고해야 하는 기본 작업 방향
- 응답 상세도: AI가 간단히 답할지, 자세히 설명할지에 대한 기본값
- 대화 세션 보관 정책: 대화 세션 저장, 삭제, 휴지통 처리 기준
- 오버레이 튜토리얼 다시 보기: AI Workspace 사용 안내 오버레이를 다시 여는 기능

23.12 Danger Zone

Danger Zone은 프로젝트에 큰 영향을 주는 고위험 작업을 한 곳에서만 관리하는 영역이다.

포함 항목:

- 워크스페이스에서 프로젝트 제거
- 연결된 GitHub 레포 삭제
- 클라우드 리소스 정리
- 모든 배포 비활성화
- 프로젝트 영구 삭제

위험 작업은 General, Repository, Cloud Infrastructure 등에 흩어두지 않고 Danger Zone에서만 처리한다.

24. 프로젝트 제거 및 삭제 정책

24.1 프로젝트 제거

사용자는 Overview 또는 Project Settings의 Danger Zone에서 프로젝트를 제거할 수 있다.

기본적으로 프로젝트 제거는 서비스의 Projects 목록에서만 제거하는 동작이다.

24.2 GitHub 레포 삭제 옵션

프로젝트 제거 시 사용자가 선택하면 연결된 GitHub 레포까지 삭제할 수 있다.

GitHub 레포 삭제는 민감한 작업이므로 별도 확인과 명확한 경고가 필요하다.

24.3 대화 세션 처리

프로젝트가 제거되면 해당 프로젝트에 속한 대화 세션은 계정 전체 공용 휴지통으로 이동한다.

연결 가능한 프로젝트가 없는 대화 세션은 복구할 수 없다.

24.4 동일 레포 재불러오기

동일한 GitHub 레포를 다시 불러오면 새 프로젝트처럼 프로젝트 목록에 추가될 수 있다.

기존에 휴지통으로 이동한 대화 세션은 자동 복구하지 않는다.

대화 세션 복구는 별도 동작으로만 가능하다.

25. 빌드 실패 및 오류 처리

25.1 실패 처리 원칙

Docker 기반 미리보기 또는 배포 전 빌드 과정에서 실패가 발생하면, 시스템은 단순 실패 메시지만 보여주지 않는다.

시스템은 실패 원인과 에러 원인을 분석해야 한다.

25.2 사용자에게 보여줄 정보

사용자에게는 다음을 함께 보여준다.

- 쉬운 설명
- 원인 로그 일부
- 가장 가능성이 높은 최선의 수정안 1개

25.3 수정 흐름

사용자가 수정안을 승인하면 코드 관련 Agent가 수정 후 다시 빌드와 미리보기를 진행한다.

26. 지원 프로젝트 유형

Qeploy는 초기에는 정적 프론트엔드와 React 프로젝트를 자연스럽게 지원하지만, 전체 제품 범위에서는 프론트엔드, 백엔드, 풀스택 프로젝트까지 지원한다.

지원 방향:

- 정적 HTML/CSS/JS
- React
- 프론트엔드 프로젝트
- 백엔드 프로젝트
- 풀스택 프로젝트
- API 서버
- 서버리스 함수
- 데이터베이스가 필요한 웹서비스

Docker 기반 미리보기와 AWS/GCP 배포 구조를 사용하기 때문에, 최종적으로 정적 사이트뿐 아니라 실제 서버와 데이터베이스가 필요한 웹서비스까지 다룰 수 있어야 한다.

27. 성공 지표

Qeploy의 성공은 단순히 AI가 코드를 잘 생성하는지가 아니라, 사용자가 요청한 웹서비스가 실제로 실행되고, 검증되고, 배포되고, 운영 가능한 상태까지 도달했는가로 판단한다.

27.1 Docker 미리보기 성공률

사용자의 요청 결과가 Docker 컨테이너에서 정상 설치·빌드·실행되는 비율

27.2 배포 성공률

승인 후 GitHub Pages / AWS / GCP 배포가 정상 완료되는 비율

27.3 빌드 실패 해결률

빌드 실패 발생 시 원인 분석과 수정 제안을 통해 해결되는 비율

27.4 요청 처리 정확도

사용자의 자연어 요청이 코드 / 배포 / 도메인 / 인프라 운영 작업으로 정확히 분류되고 처리되는 비율

27.5 도메인 연결 성공률

geploy.com 서브도메인 또는 사용자 지정 도메인이 정상 연결되는 비율

27.6 클라우드 예상 요금 신뢰도

AWS/GCP 설정 화면의 예상 요금이 실제 비용과 얼마나 근접한지

28. UX 원칙

28.1 복잡성 숨기기

사용자는 GitHub, Docker, 배포, DNS, AWS/GCP 같은 복잡한 기술 개념을 직접 이해하지 않아도 된다.

28.2 자연어 우선

사용자는 자연어로 원하는 결과를 말한다.

시스템은 Agent 구조와 자동화 workflow로 작업을 처리한다.

28.3 미리보기 우선

사용자는 실제 반영 전 Docker 기반 미리보기에서 동작 결과를 확인할 수 있어야 한다.

28.4 승인 기반 통제

사용자에게 영향을 주는 실제 작업은 승인창을 통해 확인 후 진행한다.

28.5 한 줄 요약 중심

승인창과 주요 상태 정보는 기술적 상세보다 사용자가 이해 가능한 한 줄 요약 중심으로 제공한다.

28.6 결과 중심 정보 제공

사용자가 반드시 알아야 하는 정보는 다음으로 제한한다.

- 미리보기 결과
- 승인창의 작업 요약
- 배포 URL

- 현재 버전
 - 예상 비용
-

29. 최종 요약

Qeploy는 비개발자와 입문 개발자가 AI Agent와 자연어로 웹 프로젝트를 생성·수정하고, Docker 기반 미리보기에서 실제 동작을 검증한 뒤, GitHub 기반 이력/버전 관리를 거쳐 GitHub Pages 또는 개인 AWS/GCP 같은 배포 환경에 배포하고, qeploy.com 서브도메인 또는 사용자 지정 도메인까지 연결할 수 있게 하는 AI Agent 기반 웹서비스 운영 자동화 플랫폼이다.

Qeploy의 핵심은 단순 코드 생성이 아니라, 웹서비스가 실제로 운영 가능한 상태에 도달하기까지 필요한 개발, 검증, 버전관리, 배포, 도메인, 인프라 설정, 운영 관리를 하나의 자연어 기반 흐름으로 연결하는 것이다.